

VulnPulse

A Cyber Vulnerability Intelligence Data Pipeline

Phase 1 Project Proposal

Data Analysis and Visualization

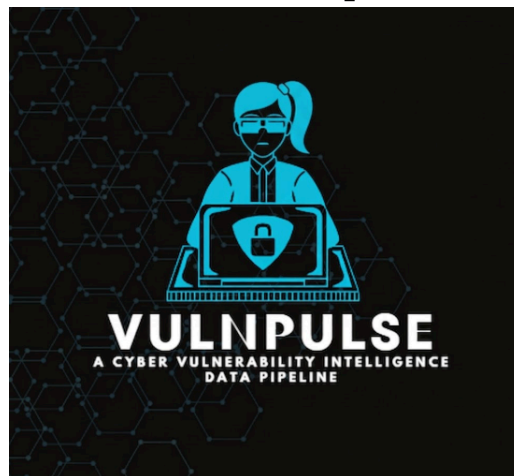
Submitted by

Kashish Fatima — 24L-2605

Zahra Saeed — 24L-2512

GitHub Repository: <https://github.com/kashishfatima999/VulnPulse>

Submission Date: 25 September 2026



1. Domain and Source Identification

For this project we are working in the domain of cybersecurity vulnerability intelligence. Every year tens of thousands of new CVEs are published, but knowing that a vulnerability exists is not the same as knowing whether it actually matters.

The gap. NVD tells you what is vulnerable, but not what is actually being exploited. CISA's KEV catalog tells you what is being exploited, but on its own it is a fairly thin list without much surrounding context. Security teams end up checking both sources separately and manually connecting the dots, which does not scale once you are dealing with thousands of records.

Our approach. VulnPulse closes this gap by combining the two sources into one pipeline. NVD serves as the core vulnerability dataset, enriched with CISA KEV exploitation data, and both are processed through a Bronze, Silver, and Gold pipeline so the final output is not raw records but actual answers: which vendors carry the most exploited vulnerabilities, which weakness categories show up most often among actively exploited CVEs, and how long it typically takes before a published vulnerability is confirmed as exploited in the wild.

Why Spark. The use of Spark is deliberate rather than incidental. The project involves nested JSON structures, large historical volumes, multi-source joins, and a genuine full-load-plus-incremental-load pattern, which is exactly the kind of workload distributed processing is meant for.

Our primary data source is the National Vulnerability Database (NVD), maintained by NIST. NVD publishes vulnerability records (CVEs) along with severity scores, affected products, and weakness classifications. We are using two parts of NVD: the yearly JSON 2.0 feeds for our historical baseline, and the NVD CVE API 2.0 for ongoing updates. We are also pulling in the CISA Known Exploited Vulnerabilities (KEV) catalog as a second source, since it tells us which of these vulnerabilities are known to have been exploited in the real world, which NVD alone does not tell us.

Exact references we are using:

- NVD website: <https://nvd.nist.gov/>
- NVD yearly JSON 2.0 feeds (full load): <https://nvd.nist.gov/feeds/json/cve/2.0/>
- NVD CVE API 2.0 (incremental load): <https://services.nvd.nist.gov/rest/json/cves/2.0>
- NVD developer documentation: <https://nvd.nist.gov/developers/vulnerabilities>
- CISA KEV catalog: <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- CISA KEV JSON feed:
https://www.cisa.gov/sites/default/files/feeds/known_exploited_vulnerabilities.json

1.1 Full Load and Incremental Load

Both of our data sources support the two ingestion patterns required for this project.

For the full load, we will download NVD's yearly JSON 2.0 feeds covering 2020 through 2026. These files follow a predictable naming pattern (nvdcve-2.0-{YEAR}.json.gz) and together they give us a large historical baseline to build the initial Bronze and Silver tables from.

For the incremental load, we will use the NVD CVE API with its lastModStartDate and lastModEndDate parameters. This lets us ask NVD for only the records that changed after a given timestamp, instead of downloading everything again. We will keep track of the last timestamp we successfully processed (a watermark) and use that as the starting point for the next run. This is the same general pattern used in real data engineering pipelines, where you do not want to reprocess the entire history every time new data arrives, such as new daily stock prices or new commits to a repository.

CISA KEV will be refreshed periodically as a smaller enrichment snapshot rather than treated as a full/incremental pair on its own, since the catalog itself is comparatively small and is joined against the NVD records rather than ingested independently.

2. Data Samples and Volume

We have already pulled sample files from both load types and pushed them to our GitHub repository under data/samples/, rather than just describing what they would look like. The samples include ten records extracted from a real NVD 2.0 yearly feed (representing the full load), ten records pulled from a live NVD API query filtered by lastModified date (representing an incremental load), and a small set of current CISA KEV entries. Each sample is kept under one megabyte, and a short manifest file records where each sample came from, when it was captured, and how many records it contains, so the samples can be traced back to their source.

2.1 Volume Estimation

The full NVD historical baseline (2020-2026) is a few hundred megabytes when compressed, based on the sizes published for the yearly feeds. We are not committing this to GitHub since it is too large and not needed for version control; it will only be pulled locally or into Databricks when the pipeline actually runs. We expect the incremental daily update to be much smaller, typically well under a megabyte to a few megabytes depending on how many CVEs were modified that day. The CISA KEV snapshot is around 1-2 MB as a whole and can simply be re-downloaded each time it is refreshed rather than accumulated.

Component	Approximate size	Notes
NVD full baseline (2020-2026)	~130 MB compressed	Not stored in GitHub; pulled directly during processing

Component	Approximate size	Notes
NVD daily incremental batch	Under 1 MB to a few MB	Depends on number of records modified that day
CISA KEV snapshot	~1-2 MB	Refreshed periodically, not accumulated
GitHub sample files	Under 1 MB each	Small excerpts kept for reproducibility and evidence

These numbers are estimates based on what NVD and CISA currently publish; we will record the actual measured sizes once we run the pipeline in Phase 2, since feed sizes do shift slightly over time as records are added.

3. Security and Compliance

We went through our sample data specifically looking for anything that could count as Personally Identifiable Information, meaning data that could identify a specific person, such as a name, email address, or IP address.

VulnPulse does not deal with individual users or customers, so most of the concern around PII does not apply here. The one field worth flagging is `sourceIdentifier` in the NVD records. This field records who submitted or last touched a CVE entry, and in some cases it contains an email-style address rather than an organization name. It is not something we need for our analysis, so our plan is to detect this pattern during the Bronze to Silver transformation and drop it from any table that analysts or the dashboard will actually query. If it is useful to know whether a record originally had this kind of identifier, we can keep a simple true/false flag instead of the raw value.

Everything else in the data, such as CVE identifiers, CVSS scores, vendor and product names, and CWE classifications, is public vulnerability information and does not raise a privacy concern. We will not store any API keys or credentials in the repository; those will stay in local environment variables or in Databricks secrets, kept outside of version control.

4. High-Level Medallion Data Modeling

4.1 Bronze and Silver Layers

The Bronze layer will hold the raw JSON payloads pretty much as we received them from NVD and CISA, along with some basic metadata we add ourselves: which batch it came from, which source, what time it was ingested, and a checksum so we can tell if something changed unexpectedly. The point of Bronze is just to preserve what we actually received, so if something

goes wrong downstream we can always go back and reprocess from here without having to re-fetch from NVD or CISA.

Moving from Bronze to Silver is where most of the real cleaning happens. NVD records come as deeply nested JSON, so we need to flatten them into proper columns. This involves pulling out the English-language description, converting date strings into actual timestamp types, standardizing severity labels, extracting the numeric CVSS score, and separating out the list of affected vendors and products into their own relationship table rather than leaving them nested inside each CVE record. We will also normalize CWE identifiers, remove duplicate CVE entries, and check for obviously missing or invalid values such as dates outside a reasonable range or CVSS scores outside 0-10.

At a high level, the Silver layer will be organized around five tables: a main CVE table holding one row per vulnerability, a CVE-to-product table capturing which vendors and products are affected, a CVE-to-CWE table for weakness classification, a CVE-to-reference table for external links, and a KEV table holding the exploitation status and dates pulled from CISA.

4.2 Gold Layer

The Gold layer is where we build the tables that Power BI will actually connect to, so these need to be compact and already aggregated rather than raw. We are planning six Gold tables: one tracking vulnerability counts and severity over time, one summarizing vendor and product risk exposure, one measuring the time gap between a CVE being published and being added to the KEV catalog, one tracking trends by weakness category (CWE), one acting as a current watchlist of high-priority vulnerabilities, and one that records pipeline run statistics such as record counts and rejected rows, mainly for our own debugging and to show the pipeline is actually working correctly rather than silently dropping data.

5. Business Intelligence and Dashboards

We will build the final dashboard in Power BI, connected only to the Gold tables. The dashboard is meant for someone in a security or vulnerability-management role who needs to prioritize what to look at, not for someone trying to browse raw CVE listings.

The dashboard should be able to answer a few specific questions: how vulnerability volume and severity has changed over time, which vendors and products show up most often among known-exploited vulnerabilities, which weakness categories are most common among exploited CVEs, how long it typically takes for a published vulnerability to be added to the KEV catalog, and which currently open vulnerabilities meet our own watchlist criteria.

To answer these, we are planning at least these visuals:

- A trend line showing CVE counts by month or quarter, broken down by severity.

- A bar chart ranking vendors by total and by known-exploited vulnerability count.
- A distribution chart showing how many days typically pass between a CVE's publication and its addition to the KEV catalog.

Depending on time, we may also add a CWE-based breakdown and a simple watchlist table listing current high-priority CVEs with their severity and KEV status, but the three visuals above are the minimum we are committing to for Phase 3.

6. Engineering Setup and FinOps

6.1 Platform

We are using Databricks Free Edition as our Spark environment, since it is free and sufficient for a project of this size. One thing we ran into while planning is that Free Edition restricts outbound internet access from the cluster, so we cannot call the NVD or CISA APIs directly from inside a Databricks notebook. Our plan is to handle data acquisition with a small local Python script, which downloads the feeds and API responses and uploads them to a Databricks volume, after which the Spark pipeline takes over for the Bronze, Silver, and Gold transformations.

6.2 Managing Storage and Compute

Since we are on free-tier infrastructure, we are being deliberate about not wasting storage or compute. We are not committing full NVD or CISA feeds to GitHub, only the small sample files described in Section 2. We are not keeping duplicate copies of raw data sitting around once it has landed successfully in Bronze. After the initial historical baseline is loaded, the pipeline will only reprocess records that changed, using the watermark approach described earlier, rather than reloading everything from scratch each time. When testing our Spark code, we plan to start small, first with the sample files, then with a single year of data, and only run against the full 2020-2026 baseline once we are confident the pipeline logic works. We are also avoiding `collect()` calls on large DataFrames in Spark, since that pulls everything into driver memory, and instead we will aggregate within Spark and only export the smaller Gold-level tables.

Our repository, at <https://github.com/kashishfatima999/VulnPulse>, currently contains the sample data files, this proposal, and the basic project structure. It will continue to grow through Phase 2 and Phase 3 as we add the ingestion and transformation code, but we intend to keep raw feeds, Parquet or Delta exports, and any large generated files out of version control throughout the project.

7. Timeline

Phase 1 (this submission) covers the proposal, sample data, and source documentation described above. Phase 2, due 10 October 2026, will cover building the actual ingestion pipeline:

the full and incremental loads, the Bronze and Silver transformations, and basic data-quality checks. Phase 3, due 24 October 2026, will cover the Gold layer and the finished Power BI dashboard, along with end-to-end testing of the whole pipeline. Detailed requirements for Phase 2 and Phase 3 will be shared separately, so this timeline may be adjusted once those are released.